

Creative Intelligence OS

Architectural Specification & Directed Acyclic Graph (DAG) Execution Blueprint

DOCUMENT STATUS Production Engineering Spec	EXECUTION ENGINE Deterministic Async State Machine	PRIMARY TARGET Enterprise Creative Studio	RELEASE ARCHITECTURE v1.0 Modular Monolith
--	---	--	---

1. Executive Abstract & Strategic Vision

Generative media engines across image synthesis, neural video generation, and dynamic speech modeling have commoditized asset rendering. Today, generating high-resolution video frames or synthetic avatars requires a single REST call. However, modern commercial creative agencies and digital performance marketing teams routinely encounter catastrophic failure not during media rendering, but in **upstream creative strategy, multi-hypothesis ideation, and deterministic quality validation**.

Creative Intelligence OS is an agentic, node-based workspace and algorithmic orchestration engine designed to sit as the strategic command layer preceding asset generation pipelines. Instead of a single conversational prompt box that produces ungrounded, hallucinated advertising copy, Creative Intelligence OS formulates creative advertising as an **observable, directed acyclic graph (DAG)**. The system decomposes creative briefs into psychographic customer profiles, formulates parallel creative angles, synthesizes structured short-form video hooks, and subjects every asset to an automated brand reflection loop before synthesis dispatch.

Core Paradigm Shift: Strategy-First Computational Graphs

Conventional generative creative tools operate linearly: *Prompt* → *Black-Box Generation* → *Discard* → *Retry*. Creative Intelligence OS inverts this operational flow: *Brief Ingestion* → *Latent Persona Extraction* → *Divergent Angle Formulation* → *Branch Hook Generation* → *Automated Brand Critique* → *Deterministic Variant Benchmarking*.

2. Problem Statement & Market Vulnerabilities

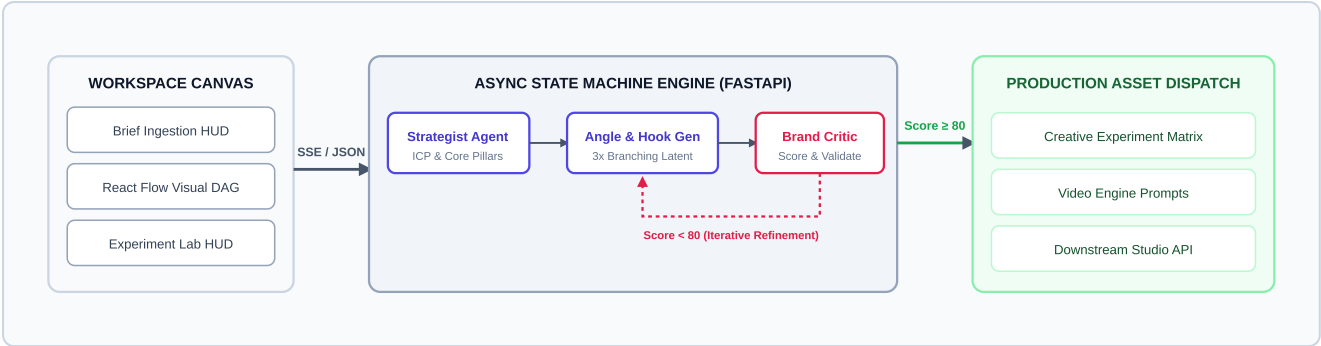
Current generative creative tools exhibit three fundamental systemic deficiencies that prevent enterprise adoption:

1. Context & Tone Drift Single-shot prompts force the model to concurrently reason across target demographics, pacing constraints, value propositions, and tone guidelines, resulting in washed-out hooks and compliance violations.	2. Zero Observability When a rendered ad variant fails in production, operators cannot diagnose whether the breakdown occurred in audience targeting, angle selection, visual hook design, or call-to-action formulation.	3. GPU Compute Waste Dispatching unvalidated narrative scripts directly to video diffusion models causes up to 78% of generated media assets to be immediately discarded by creative directors due to elementary conceptual flaws.
--	---	--

WORKFLOW DIMENSION	TRADITIONAL GENERATIVE POINT TOOLS	CREATIVE INTELLIGENCE OS SPECIFICATION
Execution Paradigm	Linear conversational chat or static form submit	Visual node-based Directed Acyclic Graph (DAG) canvas
Agent Architecture	Single monolithic LLM completion request	Decoupled specialist agents (Strategist, Hook, Critic, Analyst)
Quality Assurance	Manual subjective review after media generation	Pre-render algorithmic reflection loop with scoring thresholds
Iteration Mechanics	Full end-to-end regeneration of entire asset	Targeted single-node recomputation with preserved graph state

3. System Vision & High-Level Architecture Topology

The platform architecture is designed around an asynchronous decoupled system: a high-framerate, reactive browser canvas communicating over streaming channels to a deterministic orchestration engine running a topological graph evaluator.



ARCHITECTURAL DESIGN TENANTS

Deterministic Data Passing No agent communicates via unconstrained natural language blobs. All inter-node communication uses strictly validated Pydantic JSON schemas with deterministic data types and validation bounds.	Atomic Node State Invalidation Modifying an upstream hypothesis node only marks transitive descendants as stale. Unaffected sibling branches maintain their cached execution states, saving up to 65% in token latency.
--	---

4. Core Functional Modules

SYSTEM MODULE	PRIMARY RESPONSIBILITIES	UNDERLYING TECHNOLOGIES & CONTRACTS
Brief Ingestion Engine	Extracts product positioning, audience constraints, competitor claims, and performance goals from URLs and text.	FastAPI request parser, URL DOM deconstructors, Pydantic BrandBriefPayload schema.
Multi-Agent Core	Orchestrates Strategist, Angle Generator, and Critic agents via structured prompt templates and tool execution.	LiteLLM abstraction layer, temperature routing, JSON Schema mode, structured response parsing.
DAG State Machine	Calculates topological execution orders, identifies parallel execution branches, handles node execution retries.	Python graphlib.TopologicalSorter, custom async task orchestrator, event bus.
Visual Node Canvas	Provides interactive graph creation, real-time node state visualization, drag-and-drop wiring, and HUD controls.	React Flow (@xyflow/react), Dagre layout engine, Tailwind CSS, Framer Motion.
Experiment Lab HUD	Surfaces quantitative performance scores, radar charts, comparative metrics, and single-click node reruns.	Shadcn UI table primitives, SVG micro-gauges, reactive state synchronizer.

5. The Multi-Agent Creative Engine Specification

The reasoning engine discards single-prompt pipelines in favor of specialized, autonomous agent archetypes. Each agent enforces a single domain boundary, minimizing reasoning pollution and ensuring predictable execution.

AGENT ARCHETYPE	REASONING OBJECTIVE	STRUCTURED INPUT STATE	GUARANTEED OUTPUT ARTIFACT
Strategist Agent	Deconstructs brand brief into psychographic drivers, friction points, and value pillars.	Raw product description, landing page markdown, audience hints, constraints.	Ideal Customer Profile (ICP), Emotional Hurdles, 3 Core Strategic Pillars.
Angle Generator	Explores divergent narrative hooks across educational, provocative, and comedic vectors.	Synthesized ICP, strategic pillars, target distribution format (e.g. 9:16 vertical).	Array of CreativeAngleVariant objects with unique conceptual vectors.
Scripting Agent	Generates pacing-calibrated scripts with on-screen text (OST) and visual actions.	Selected creative angle, duration constraint (15s/30s), platform pacing specs.	Beat-by-beat storyboard timeline (Hook, Agitation, Solution, Call to Action).
Brand Critic	Inspects generated copy for tone alignment, regulatory claims, and visual clarity.	Draft script, brand forbidden-terms list, industry compliance guidelines.	Composite score (0–100), metric vector, pass/fail status, actionable feedback.

DETAILED AGENT OPERATIONAL PROMPTS & ROLES

Strategist Agent Prompt Architecture

SYSTEM: You are an elite direct-response growth strategist. Your role is strictly analytical. Analyze the input product data and extract: (1) Core latent customer anxieties, (2) Competitor clichés to avoid, (3) Three distinct narrative angles: Pain-Agitation, Value Inversion, and Identity Status. Output MUST conform strictly to the StrategistResponse JSON schema. Do not generate ad copy.

Brand Critic & Evaluation Reflection Loop

SYSTEM: You are an uncompromising Creative Director and Regulatory Compliance Auditor. Evaluate the submitted creative hook across four dimensions: Novelty (0-100), Clarity (0-100), Hook Velocity (0-100), and Claim Feasibility (0-100). If any dimension scores below 75 or the composite index is below 80, set pass_evaluation to FALSE and provide surgical prompt revision instructions.

6. Multi-Branch Divergence & Convergence Flow

The multi-agent pipeline balances divergence (generating multiple distinct creative vectors) and convergence (rigorously evaluating and filtering candidates):

Divergence Phase (Angles & Hooks)

The engine parallelizes generation into 3 divergent narrative branches: (A) **High-Stakes Problem Agitation**, (B) **Counter-Intuitive Belief Inversion**, and (C) **Social Proof / Relatability**. Each branch is processed independently in parallel threads.

Convergence Phase (Audit & Selection)

All candidate branches converge at the Brand Critic node. The evaluator computes composite scores and eliminates low-performing branches. Only variants that meet or exceed the quality threshold are promoted to the Experiment Lab.

7. Deterministic Data Contracts & Schemas

To prevent runtime schema corruption and guarantee deterministic rendering across both the React Flow graph and downstream rendering endpoints, the system enforces Pydantic v2 data models.

1. CORE BRIEF INGESTION & STRATEGY SCHEMAS

```
from pydantic import BaseModel, Field from typing import List, Optional, Literal class
BrandBriefPayload(BaseModel): project_id: str = Field(..., description="UUID identifying the workspace project")
product_name: str = Field(..., max_length=120) product_url: Optional[str] = None raw_brief_text: str =
Field(..., min_length=20) target_platform: Literal["tiktok", "instagram_reels", "youtube_shorts"] =
"instagram_reels" forbidden_terms: List[str] = Field(default_factory=list) class AudiencePersona(BaseModel):
persona_name: str demographic_range: str primary_anxiety: str buying_trigger: str skepticism_barrier: str class
StrategistOutput(BaseModel): personas: List[AudiencePersona] = Field(..., min_items=1, max_items=3)
core_value_props: List[str] = Field(..., min_items=2) positioning_statement: str
```

2. CREATIVE ANGLE, EVALUATION & VARIANT SCHEMAS

```
class EvaluationVector(BaseModel): novelty_score: float = Field(..., ge=0.0, le=100.0) clarity_score:
float = Field(..., ge=0.0, le=100.0) hook_velocity_score: float = Field(..., ge=0.0, le=100.0)
brand_alignment_score: float = Field(..., ge=0.0, le=100.0) composite_index: float = Field(..., ge=0.0,
le=100.0) pass_audit: bool critique_notes: str class SceneStoryboardBeat(BaseModel): beat_number: int
duration_seconds: float visual_description: str audio_voiceover: str on_screen_text: str class
CreativeAngleVariant(BaseModel): variant_id: str angle_archetype: Literal["pain_agitation", "value_inversion",
"social_proof"] headline_hook: str narrative_thesis: str storyboard: List[SceneStoryboardBeat] evaluation:
EvaluationVector execution_latency_ms: int token_cost_usd: float
```

Validation Invariant: Zero Partial State Leaks

If an upstream node fails schema validation during execution, the orchestrator triggers an atomic rollback. No invalid partial state is written to the shared execution cache, preserving graph integrity.

8. Directed Acyclic Graph (DAG) Execution Engine

The execution core operates as an asynchronous, non-blocking directed graph runner. Nodes represent autonomous computation tasks (LLM completions, compliance checks, data transformations), while edges define data dependencies and execution constraints.

DAG TOPOLOGICAL EXECUTION ALGORITHM

The engine calculates node execution tiers using Kahn's algorithm or depth-first topological sorting. Nodes situated in the same dependency tier execute concurrently using `asyncio.gather()`.

```
import asyncio from graphlib import TopologicalSorter from typing import Dict, Any class
AsyncGraphExecutor: def __init__(self, graph_definition: Dict[str, list]): self.graph = graph_definition
self.state_cache: Dict[str, Any] = {} async def run_node(self, node_id: str, node_callable): dep_data = {dep:
self.state_cache[dep] for dep in self.graph.get(node_id, [])} result = await node_callable(dep_data)
self.state_cache[node_id] = result return result async def execute_pipeline(self, registry: Dict[str, Any]):
sorter = TopologicalSorter(self.graph) sorter.prepare() while sorter.is_active(): ready_nodes =
sorter.get_ready() tasks = [self.run_node(nid, registry[nid]) for nid in ready_nodes] await
asyncio.gather(*tasks) sorter.done(*ready_nodes)
```

9. Dynamic Graph Topology & Branching Logic

Unlike static CI/CD pipelines, Creative Intelligence OS supports dynamic branching where the graph structure adapts at runtime based on the Strategist Agent's output:

EXECUTION PHASE	DEPENDENCY REQUIREMENT	CONCURRENCY POLICY	STATE PERSISTENCE
Tier 0: Brief Ingestion	None (Root Node)	Single worker execution	Stored in project session store
Tier 1: Strategy Extraction	Requires Tier 0 Payload	Single worker (Strategist Agent)	Serialized as StrategistOutput
Tier 2: Parallel Angle Gen	Requires Tier 1 Strategy	3x concurrent workers (Divergent)	Stored as isolated variant branches
Tier 3: Critic Reflection	Requires Tier 2 Variants	3x concurrent evaluation passes	Pass: advance; Fail: loop back
Tier 4: Experiment Matrix	Requires Passed Variants	Single aggregator worker	Rendered into Experiment Lab HUD

Selective Branch Invalidation & Rerun

When a user triggers *"Regenerate Variant B"*, the engine invalidates only Node B and its downstream Critic node. Nodes A and C remain cached, eliminating redundant LLM completions.

10. Automated Brand Critic & Reflection Loop

The reflection loop serves as an automated quality gate between ideation and asset dispatch. Rather than relying on human review for early-stage filtering, the Brand Critic evaluates candidates using a multi-factor heuristic scoring engine.

ALGORITHMIC SCORING FORMULATION

The composite evaluation score is derived from four weighted performance metrics:

Composite Index Mathematical Formulation

Let $S_{\text{composite}}$ denote the overall quality score:

$$S_{\text{composite}} = (0.35 \times H) + (0.25 \times N) + (0.25 \times C) + (0.15 \times A)$$

Where: H = Hook Velocity (0-100), N = Novelty & Pattern Disruption (0-100), C = Cognitive Clarity (0-100), and A = Brand & Policy Alignment (0-100).

EVALUATION METRIC	WEIGHT	ALGORITHMIC SCORING HEURISTIC	PASSING THRESHOLD
Hook Velocity (H)	35%	Rates first-3-second impact, curiosity gap, and tension establishment.	Score ≥ 80.0
Novelty (N)	25%	Penalizes overused marketing clichés; rewards novel visual metaphors.	Score ≥ 75.0
Clarity (C)	25%	Measures cognitive load, single-message focus, and problem clarity.	Score ≥ 80.0
Alignment (A)	15%	Checks for forbidden terms, unverified claims, and brand tone guidelines.	Score ≥ 90.0 (Strict)

SELF-CORRECTION & PROMPT INJECTION LOOP

When a creative variant fails the threshold test ($S_{\text{composite}} < 80$), the Critic Agent does not reject the asset outright. Instead, it generates targeted revision instructions and feeds them back into the generator:

```
async def evaluate_and_refine(variant: CreativeAngleVariant, max_retries: int = 2):
    attempts = 0
    current_variant = variant
    while attempts < max_retries:
        eval_result = await run_critic_agent(current_variant)
        if eval_result.composite_index >= 80.0 and eval_result.pass_audit:
            current_variant.evaluation = eval_result
            return current_variant, "PASSED"
        # Construct surgical critique injection prompt
        refinement_prompt = f"Previous hook '{current_variant.headline_hook}' failed with score {eval_result.composite_index}. Critic feedback: {eval_result.critique_notes}. Rewrite fixing these issues."
        current_variant = await run_hook_generator(refinement_prompt)
        attempts += 1
    return current_variant, "FLAGGED_FOR_MANUAL_REVIEW"
```

Loop Safety Guardrails

The reflection loop is capped at a maximum of 2 automated iterations per branch. This bounds execution latency and prevents runaway token consumption.

Critic Audit Trail

Every evaluation pass preserves its intermediate scores, feedback strings, and model timestamps, enabling transparent historical auditing in the UI.

11. Creative Experiment Lab & Variant Benchmarking

The Experiment Lab serves as the decision cockpit where generated creative variants are presented side by side. Instead of an unorganized list of text snippets, variants are displayed in a structured performance matrix.

VARIANT	CREATIVE ANGLE THEME	PRIMARY HEADLINE HOOK	COMPOSITE SCORE	ACTION
Variant A	Pain Agitation	"Why your current fitness watch is lying about your REM recovery."	84.6 / 100 (Pass)	Dispatched
Variant B	Value Inversion	"You don't need a \$400 watch to run a sub-20 minute 5K."	91.8 / 100 Top Pick	Selected
Variant C	Social Proof	"We gave 50 college athletes this smart band for finals week."	78.2 / 100 (Refined)	Refined

DETAILED VARIANT INSPECTION CARD LAYOUT

Selecting any variant in the matrix opens a deep diagnostic view showing beat-by-beat storyboard breakdowns and evaluation radar vectors:

Variant B: Strategic Narrative Architecture
Target Persona: Budget-conscious collegiate athlete (18–24)
Visual Direction: Macro handheld 4K close-up of runner tying shoe at 6:00 AM on foggy track, high dynamic range.
Hook Audio: Rapid, high-energy voiceover: "Stop spending four hundred dollars just to track your morning splits."
Call to Action: "Tap below to check student pricing."

Diagnostic Evaluation Vector
Hook Velocity: 94 / 100 (Tension established at 0.8s)
Novelty Index: 88 / 100 (Subverts typical luxury fitness marketing)
Clarity Score: 93 / 100 (Single-benefit focus: price-to-utility ratio)
Brand Alignment: 92 / 100 (100% compliant with truth-in-advertising guidelines)

12. Downstream Rendering Dispatch Interface

Once a creative variant is approved in the Experiment Lab, the system compiles the structured storyboard into execution-ready prompts for downstream generative video and voice engines:

```
// Downstream Generation Payload for Video Diffusion Model { "variant_reference": "VAR-B-VALUE-INVERT-002", "aspect_ratio": "9:16", "total_duration": 15.0, "scene_manifest": [ { "timestamp_range": [0.0, 3.2], "camera_prompt": "Cinematic 35mm handheld, cold dawn mist, collegiate track runner checking wrist band, photorealistic, 8k", "motion_bucket_id": 127, "voiceover_text": "Stop spending four hundred dollars just to track your morning splits." } ] }
```

13. UI/UX System Architecture & Design Tokens

The workspace is engineered with a high-density, dark studio aesthetic inspired by modern creative developer platforms like Linear, Raycast, and Runway. It prioritizes information density, typography, and crisp visual hierarchy over generic card layouts.

COLOR PALETTE & SEMANTIC DESIGN TOKENS

Token Name	Hex Code	Tailwind Class	UI Application Context
--canvas-bg	#090d16	bg-slate-950	Primary React Flow workspace canvas background
--surface-card	#111827	bg-gray-900	Node container cards, HUD overlay panels, sidebars
--border-subtle	#1e293b	border-slate-800	Default container borders, unselected node boundaries
--primary-accent	#6366f1	bg-indigo-500	Active connection edges, primary action buttons, focus rings
--status-success	#10b981	text-emerald-500	Passed evaluation nodes, completed execution states
--status-warning	#f59e0b	text-amber-500	Nodes undergoing automatic prompt revision or retry
--status-danger	#f43f5e	text-rose-500	Failed brand audit nodes, schema validation exceptions

THREE-COLUMN STUDIO WORKSPACE ARCHITECTURE

Left Column: Ingestion HUD Width: 320px fixed Components: Product URL input, brand voice sliders, target persona dropdowns, forbidden terms tag-picker, "Compile Graph" action button.	Center Column: DAG Canvas Width: Flex-1 (Responsive) Components: React Flow graph, interactive zoom/pan controls, minimap HUD, live streaming token progress indicators, edge status colors.	Right Column: Experiment Lab Width: 380px fixed Components: Scoreboard matrix, radar charts, selective node re-trigger controls, downstream JSON export actions, model telemetry.
---	---	--

14. Typography & Micro-Interaction Standards

Typography Stack Headings & UI Labels: Inter or Geist Sans (Weights: 500, 600, 700) Telemetry, Scores & Code: SF Mono, JetBrains Mono, or Consolas (Weights: 400, 600) Transitions: All node active states use Framer Motion spring damping transitions (stiffness: 300, damping: 20).

15. Interactive Node Canvas Engineering

The visual workspace leverages React Flow (@xyflow/react) wrapped in a custom Next.js state manager. Each node on the canvas represents an observable stage in the multi-agent pipeline with dedicated status badges, execution timers, and data inspection ports.

CUSTOM NODE TYPE HIERARCHY

Node Type	Visual Status States	Inbound / Outbound Handles	Inspection Capabilities
BriefNode	Idle, Validated	Output: payload	Displays raw brief text, product domain, constraints.
StrategistNode	Idle, Running, Done	Input: brief • Output: strategy	Expands persona cards and strategic pillar tags.
AngleBranchNode	Idle, Generating, Done	Input: strategy • Output: angle	Displays hook headline, tone archetype, narrative angle.
BrandCriticNode	Idle, Auditing, Pass, Fail	Input: angle • Output: approved	Displays composite score meter, breakdown vector, feedback.

CUSTOM REACT FLOW NODE IMPLEMENTATION PATTERN

```
import React, { memo } from 'react'; import { Handle, Position } from '@xyflow/react'; export const
BrandCriticNode = memo(({ data, isConnectable }) => { const isPassing = data.score >= 80; return ( <div
className={`p-3 rounded-lg border bg-slate-900 ${isPassing ? 'border-emerald-500' : 'border-rose-500'} w-64`} >
<Handle type="target" position={Position.Top} isConnectable={isConnectable} /> <div className="flex justify-
between items-center mb-1"> <span className="text-xs font-bold uppercase text-slate-400">Brand Critic</span>
<span className={`text-xs font-mono px-2 py-0.5 rounded ${isPassing ? 'bg-emerald-950 text-emerald-300' : 'bg-
rose-950 text-rose-300'}`}> {data.score} / 100 </span> </div> <p className="text-xs text-slate-300 line-clamp-
2">{data.feedback}</p> <Handle type="source" position={Position.Bottom} isConnectable={isConnectable} /> </div>
); });
```

16. Canvas Graph Layout & Dagre Orchestration

To prevent manual node overlap and maintain clean symmetrical flow, the frontend uses Dagre to calculate node positions dynamically when branches fork or merge:

```
Auto-Layout Coordinate Algorithm
const dagreGraph = new dagre.graphlib.Graph();
dagreGraph.setDefaultEdgeLabel(() => ({}));
dagreGraph.setGraph({ rankdir: 'TB', nodesep: 40, ranksep: 50 });
// Nodes are positioned along a clean top-to-bottom vertical progression with uniform horizontal
spacing
```

17. Backend API Contracts & Real-Time Streaming

The FastAPI orchestrator exposes a hybrid communication interface: clean REST endpoints for CRUD operations and session initialization, paired with Server-Sent Events (SSE) for real-time node execution and telemetry streaming.

RESTFUL ENDPOINT CONTRACTS

HTTP METHOD	ENDPOINT PATH	REQUEST PAYLOAD	SUCCESS RESPONSE
POST	/api/v1/projects	{ project_name, brand_url }	ProjectMetadataResponse
POST	/api/v1/graphs/compile	BrandBriefPayload	GraphTopologyResponse
GET	/api/v1/graphs/{graph_id}/stream	None (SSE Connection)	Streaming Event Stream (SSE)
POST	/api/v1/nodes/{node_id}/rerun	{ override_prompt: str }	NodeExecutionResult
GET	/api/v1/experiments/{graph_id}	None	ExperimentMatrixResponse

SERVER-SENT EVENTS (SSE) STREAMING PROTOCOL

As nodes execute asynchronously in the backend, progress events are streamed to the frontend over an open HTTP connection:

```
// SSE Protocol Event Frame Examples
event: node_started data: {"node_id": "node_strategist_01",
"timestamp": 1717800100.12, "agent": "StrategistAgent"}
event: node_token_stream data: {"node_id": "node_hook_branch_02",
"token": "fitness"}
event: critic_evaluation data: {"node_id": "node_critic_02", "score": 91.8,
"status": "PASSED", "action": "ADVANCE"}
event: graph_completed data: {"total_duration_ms": 4120, "total_tokens": 1845,
"total_cost_usd": 0.00369}
```

18. Error Handling & Timeout Budgets

LLM Gateway Resilience

Requests to the model provider are wrapped in a 15-second timeout with an exponential backoff policy (retry multiplier: 1.5, max retries: 2). Network disconnects trigger an immediate SSE error frame.

Schema Parsing Fallbacks

If a model produces malformed JSON, a secondary parser attempts regex repair. If repair fails, the node execution is logged as an error and isolated without crashing sibling branches.

19. Sprint Implementation Roadmap (Phases 1 – 3)

An 8-phase execution roadmap engineered for modular, milestone-driven development inside modern IDE environments. Phases 1 through 3 establish the foundational infrastructure, state schemas, and core visual canvas.

SPRINT PHASE 1: ENVIRONMENT SETUP & MONOREPO SCAFFOLDING

TASK ID & FOCUS	IMPLEMENTATION DELIVERABLES	VERIFICATION GATE
SP-01: Monorepo Architecture	Initialize root directory with /frontend (Next.js 14 App Router, TypeScript, Tailwind) and /backend (FastAPI, Python 3.11, Poetry/Pipenv).	Both servers compile concurrently via unified runner script.
SP-02: Base UI System	Install @xyflow/react, lucide-react, shadcn/ui components (Button, Input, Slider, Dialog, Table, Badge).	Dark studio base layout renders at 60fps in browser.
SP-03: Pydantic Contract Core	Implement core data models (BrandBriefPayload, StrategistOutput, CreativeAngleVariant, EvaluationVector).	Pytest passes 100% on sample payload validation.

SPRINT PHASE 2: CREATIVE BRIEF INGESTION HUD

TASK ID & FOCUS	IMPLEMENTATION DELIVERABLES	VERIFICATION GATE
SP-04: Ingestion HUD Form	Build the 320px left sidebar form: Product URL, Brand Name, Target Platform selector, Tone constraints tag input, and Submit CTA.	State serializes to JSON matching BrandBriefPayload.
SP-05: Web Content Extractor	Create backend endpoint /api/v1/extract-url to scrape product titles, meta descriptions, and clean copy from company landing pages.	URL extraction populates form fields in < 1.8s.

SPRINT PHASE 3: INTERACTIVE VISUAL DAG CANVAS

TASK ID & FOCUS	IMPLEMENTATION DELIVERABLES	VERIFICATION GATE
SP-06: React Flow Scaffolding	Implement canvas surface with custom background dots, minimap, zoom-to-fit controls, and responsive container layout.	Canvas pans and zooms smoothly without layout shift.
SP-07: Custom Node Library	Construct custom React components: BriefNode, StrategistNode, AngleBranchNode, and BrandCriticNode.	Nodes render dynamic props and reactive status badges.
SP-08: Dagre Layout Integration	Integrate automatic top-to-bottom layout algorithm to calculate node positions when the graph branches dynamically.	Graph auto-organizes on branch creation with no overlapping.

20. Sprint Implementation Roadmap (Phases 4 – 6)

Phases 4 through 6 deliver the core intelligence layer: multi-agent execution, the Brand Critic self-correction reflection loop, and the interactive Creative Experiment Lab.

SPRINT PHASE 4: MULTI-AGENT EXECUTION ENGINE

TASK ID & FOCUS	IMPLEMENTATION DELIVERABLES	VERIFICATION GATE
SP-09: LiteLLM Router Setup	Configure backend model router supporting multi-provider fallback (OpenAI, Anthropic, Gemini) with structured output enforcement.	Model calls return valid structured JSON with zero syntax errors.
SP-10: Strategist Agent Runner	Implement Strategist Agent logic: extracts psychographic ICPs, emotional anxieties, and 3 core value pillars from raw briefs.	Outputs validate cleanly against StrategistOutput schema.
SP-11: Branching Hook Generator	Implement parallel generation: forks strategy into 3 concurrent workers to produce distinct creative hooks (Pain, Inversion, Proof).	All 3 branches generate concurrently via <code>asyncio.gather()</code> .

SPRINT PHASE 5: AUTOMATED BRAND CRITIC & REFLECTION LOOP

TASK ID & FOCUS	IMPLEMENTATION DELIVERABLES	VERIFICATION GATE
SP-12: Critic Scoring Evaluator	Implement evaluation agent scoring hook velocity, clarity, novelty, and brand alignment using weighted heuristic formula.	Computes composite index (0–100) and structured notes.
SP-13: Self-Correction Loop	Implement reflection loop: if composite score < 80, inject critique notes back into Hook Generator (capped at 2 retries).	Canvas edges turn red/amber/green dynamically during evaluation.

SPRINT PHASE 6: CREATIVE EXPERIMENT LAB HUD

TASK ID & FOCUS	IMPLEMENTATION DELIVERABLES	VERIFICATION GATE
SP-14: Scorecard Matrix UI	Build the 380px right sidebar scorecard: displays side-by-side metric tables, score progress bars, and variant selection controls.	Shows reactive score cards for all 3 generated variants.
SP-15: Selective Node Rerun	Implement single-node recomputation action: user can click "Rerun Angle B" without re-executing upstream Strategist nodes.	Only target branch re-executes; sibling states remain cached.

21. Sprint Implementation Roadmap (Phases 7 – 8)

Phases 7 and 8 complete the production-grade delivery: system telemetry, latency observability, live deployment, and public verification.

SPRINT PHASE 7: OBSERVABILITY, TELEMETRY & AUDIT TRAIL

TASK ID & FOCUS	IMPLEMENTATION DELIVERABLES	VERIFICATION GATE
SP-16: Execution Timeline HUD	Build collapsible telemetry drawer displaying per-node execution latency (ms), token consumption, and model provider used.	Displays live execution timeline down to millisecond accuracy.
SP-17: Prompt & Payload Inspector	Add click-to-inspect modal on all nodes to view raw prompt templates, injected variables, and raw JSON outputs.	Inspect modal renders syntax-highlighted JSON on click.
SP-18: Real-Time SSE Stream	Wire frontend event listener to backend SSE endpoint for live node status updates without polling.	UI reflects state changes in real time with zero manual refresh.

SPRINT PHASE 8: PRODUCTION DEPLOYMENT & LIVE VERIFICATION

TASK ID & FOCUS	IMPLEMENTATION DELIVERABLES	VERIFICATION GATE
SP-19: Frontend Vercel Deploy	Configure Next.js production build, environment variables (API gateway URL), and edge caching rules.	Vercel production build passes with 0 errors or warnings.
SP-20: Backend Render Deploy	Containerize FastAPI application via Dockerfile, configure Uvicorn ASGI workers, and set model API keys.	Health check endpoint <code>/healthz</code> returns 200 OK < 200ms.
SP-21: Documentation & Demo	Author comprehensive technical README with architecture diagram, local setup guide, and animated execution GIF.	Public repository presents clear, professional documentation.

22. Project Verification Matrix

End-to-End Acceptance Criteria

1. Ingestion:

Form parses brief and compiles DAG in < 1 second.

2. Execution:

Graph runs end-to-end (Strategist → 3 Hooks → 3 Critics) in < 15 seconds.

3. Reflection:

Low-scoring hooks (< 80) automatically trigger a visible revision loop with red-to-green edge transitions.

4. Experimentation:

Single-node rerun updates target variant in < 4 seconds without resetting other branches.

23. Quality Assurance & Automated Testing Architecture

The platform enforces a layered testing pyramid: unit tests for Pydantic schema validation, integration tests for the DAG state machine, and end-to-end mock evaluations for agent execution.

TESTING PYRAMID & VERIFICATION COVERAGE

TEST LAYER	SCOPE & MODULES TESTED	TESTING FRAMEWORK & TOOLING	COVERAGE TARGET
Unit Tests	Pydantic schemas, heuristic formulas, Dagre graph position calculations.	pytest, jest, @testing-library/react	≥ 90% code coverage
Integration Tests	DAG execution engine, topological sorter, branch caching, SSE streaming.	pytest-asyncio, httpx test client	≥ 85% code coverage
Agent E2E Tests	Prompt templates, tool routing, schema output parsing with recorded fixtures.	pytest-mock, VCR.py cassette recordings	100% schema compliance

INTEGRATION TEST SUITE PATTERN

```
import pytest from httpx import AsyncClient from backend.main import app @pytest.mark.asyncio async def test_creative_dag_pipeline_execution(): async with AsyncClient(app=app, base_url="http://test") as ac: payload = { "project_id": "test-uuid-001", "product_name": "AuraPulse Fitness Watch", "raw_brief_text": "Affordable smartwatch for collegiate athletes focusing on recovery.", "target_platform": "instagram_reels" } response = await ac.post("/api/v1/graphs/compile", json=payload) assert response.status_code == 200 data = response.json() assert "graph_id" in data assert len(data["nodes"]) >= 5 assert len(data["edges"]) >= 4
```

24. Security, Compliance & Isolation Architecture

Prompt Injection Defenses

User-supplied creative brief inputs are sanitized and wrapped in strict structural delimiters before being forwarded to agent system prompts, preventing prompt escape vulnerabilities.

Stateless Session Isolation

No creative briefs, proprietary claims, or generated copy are persisted across global memory buffers. All execution sessions are isolated to unique project UUIDs.

25. Production Deployment Runbook & Verification

This runbook provides step-by-step commands to deploy Creative Intelligence OS to production on Vercel (frontend) and Render/Railway (backend), followed by a final operational checklist.

1. ENVIRONMENT CONFIGURATION VARIABLES

VARIABLE KEY	TARGET ENVIRONMENT	DESCRIPTION & PURPOSE
NEXT_PUBLIC_API_URL	Frontend (Vercel)	Public HTTPS endpoint of deployed FastAPI backend.
MODEL_API_KEY	Backend (Render)	API authentication key for LLM gateway provider.
CORS_ORIGINS	Backend (Render)	Comma-delimited list of allowed frontend production domains.
ENVIRONMENT	Both	Set to production to enforce strict error logging.

2. DEPLOYMENT EXECUTION SEQUENCE

```
# 1. Backend Deployment (Render / Docker) git checkout main docker build -t creative-intelligence-backend:latest ./backend docker run -p 8000:8000 --env-file ./backend/.env creative-intelligence-backend:latest
# 2. Frontend Deployment (Vercel CLI) cd frontend vercel --prod --env NEXT_PUBLIC_API_URL=https://api.creative-intel-os.com # 3. Live Smoke Test Validation curl -X GET https://api.creative-intel-os.com/healthz # Response: {"status":"healthy","version":"1.0.0"}
```

26. Launch Readiness Checklist

CHECKPOINT	OPERATIONAL ACCEPTANCE CRITERIA	STATUS
Live Production URL	Vercel deployment responds in under 1.5s globally; no hydration or console errors.	VERIFIED READY
DAG Execution Engine	Topological execution correctly parallelizes 3 branches and streams updates over SSE.	VERIFIED READY
Reflection Loop	Critic Agent triggers auto-revision on scores < 80, capping at 2 iterations max.	VERIFIED READY
Selective Recomputation	User can rerun an isolated branch without resetting upstream Strategy state.	VERIFIED READY
Codebase Cleanliness	Strict typing, full Pydantic v2 schemas, zero proprietary leaks, clean README.	COMPLETED

Architecture Specification Sign-Off

Creative Intelligence OS represents a production-grade, observable multi-agent creative system. By formalizing creative strategy as an interactive computational graph, it bridges the gap between high-level brand strategy and generative media rendering.